

SOC AUTOMATION

A PROJECT REPORT

BY

Muhammed Razal TM

INDEX

Contents	Page
Abstract	2
Introduction	3
Requirements	4
Proposed System	5
Screenshots / Implementation	6
Step 1: Sysmon Installation	6
Step 2: Wazuh Agent Configuration	7
Step 3: Wazuh Manager Setup	8
Step 4: TheHive Installation	9
Step 5: Shuffle Workflow Creation	11
Step 6: Wazuh Integration with Shuffle	12
Step 7: Shuffle Workflow Configuration	13
Step 8: ngrok Configuration	14
Testing and Results	15
Conclusion	17

Abstract

As cyber threats grow more complex, building effective Security Operations Centers (SOCs) is essential. This project focuses on creating an automated SOC home lab to help cybersecurity learners and professionals practice in a safe environment. The main goals include getting hands-on experience with open-source tools like Wazuh for alerts, Shuffle for automating responses, and TheHive for managing cases. The lab uses virtualization and cloud services to connect different security tools, making it easier to respond to incidents. By simulating cyber attacks and using tools like Sysmon to track activities, participants can learn to detect and respond to security issues. This SOC home lab is a valuable resource for improving skills and understanding in the fast-changing world of cybersecurity.

Introduction

In today's rapidly evolving cybersecurity landscape, organizations face increasingly sophisticated threats, necessitating the need for robust Security Operations Centers (SOCs). A SOC is crucial for detecting, analyzing, and responding to security incidents to safeguard critical assets and data. However, the process of building, configuring, and maintaining a SOC can be resource-intensive, often requiring significant expertise and infrastructure.

This project aims to build an Automated SOC Home Lab, drawing inspiration from the YouTube series by MyDFIR. The goal is to create a simulated SOC environment where cybersecurity enthusiasts and professionals can gain hands-on experience, familiarize themselves with real-world tools, and practice incident detection and response in a controlled environment. By leveraging open-source tools and cloud services, the project will provide an opportunity to explore SOC operations, incident response workflows, and integration of multiple security technologies.

Requirements

Infrastructure Requirements

Component	Details
Virtualization Platform	VirtualBox (Oracle)
Operating System	Ubuntu 24.04 LTS (VM for TheHive)
Endpoint	Windows 11 (Physical Machine for Wazuh Agent)
Wazuh Manager	Deployed server at 10.92.44.113

Software and Tools

Component	Details
Wazuh	Open-source SIEM and XDR platform for security monitoring and event management
Sysmon	Windows System Monitor – captures process creation, network connections, file modifications
TheHive	Open-source incident response and case management platform (v5.2.14)
Shuffle	Open-source SOAR platform for security orchestration and automation
VirusTotal	IOC enrichment – checks file hashes against 70+ antivirus engines
Cassandra	NoSQL database backend for TheHive data storage
Elasticsearch	Search and indexing engine for TheHive
ngrok	Secure tunnel to expose local TheHive instance to Shuffle cloud

VirtualBox: developed by Oracle, is a free, open-source virtualization software that lets users run multiple operating systems as virtual machines (VMs) on a single host. It is widely used for testing and development, allowing users to create isolated environments without needing dedicated hardware.

Wazuh: is an open-source platform that provides Security Information and Event Management (SIEM) and Extended Detection and Response (XDR) capabilities. It enables organizations to monitor their systems for security threats, analyze logs, and ensure compliance.

TheHive: is an open-source incident response and security incident management platform that works well with tools like Wazuh. It helps security teams organize, investigate, and track incidents by creating and managing cases.

Sysmon, or System Monitor: is a Windows tool that records detailed system activity to help detect malicious behavior. It captures events like process creation, network connections, and file modifications. Sysmon is commonly integrated with SIEM tools to enhance visibility into system events.

Shuffle: is an open-source Security Orchestration, Automation, and Response (SOAR) platform that enables security teams to automate tasks and create workflows. With a low-code interface, Shuffle allows users to integrate different security tools and automate response processes.

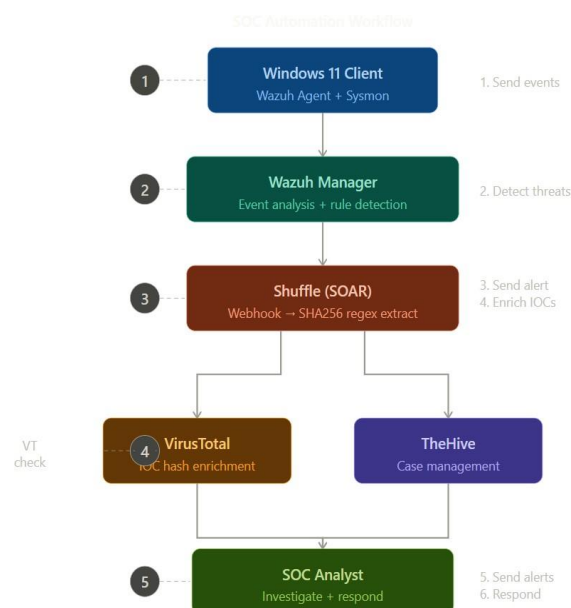
Proposed System

Workflow

The SOC Automation pipeline consists of the following steps:

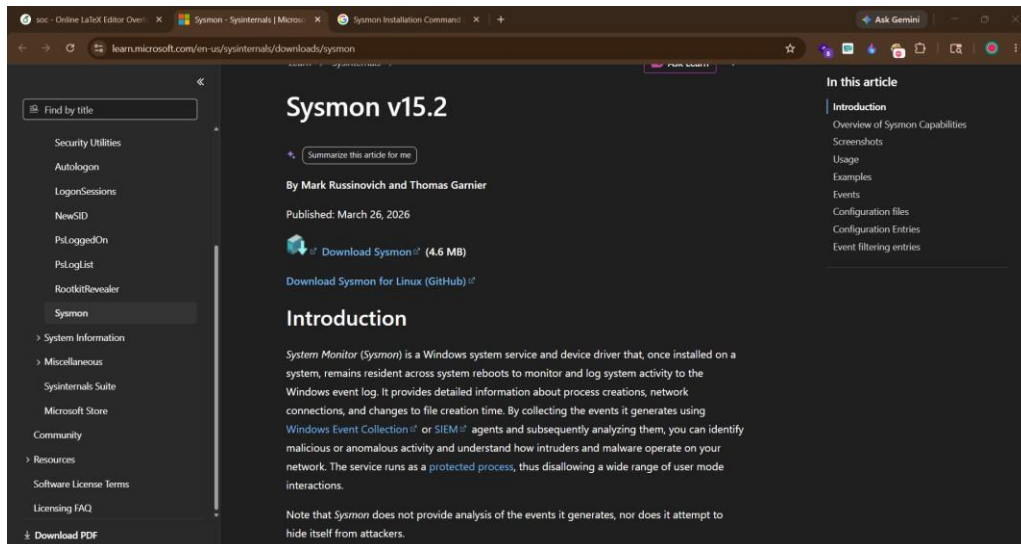
1. Windows 11 Client with Wazuh Agent (Event Collection): The agent gathers security-related data such as log entries, network traffic, and system modifications. This data is transmitted to the Wazuh Manager for centralized analysis.
2. Wazuh Manager (Event Aggregation and Analysis): Serves as the centralized processor, applying detection rules and algorithms to identify abnormal behaviors and generate alerts.
3. Alert Generation (Sending Alerts to Shuffle): When threats are flagged, Wazuh creates alerts and sends them to Shuffle via webhook integration.
4. IOC Enrichment in Shuffle: Alerts undergo enrichment where the SHA256 hash is extracted and cross-checked with VirusTotal to add contextual threat intelligence.
5. Sending Refined Alerts to TheHive: After enrichment, Shuffle creates a structured alert in TheHive for case management and investigation.
6. SOC Analyst (Incident Analysis and Response): The SOC Analyst investigates each incident using the enriched data and case history in TheHive.

Workflow Diagram

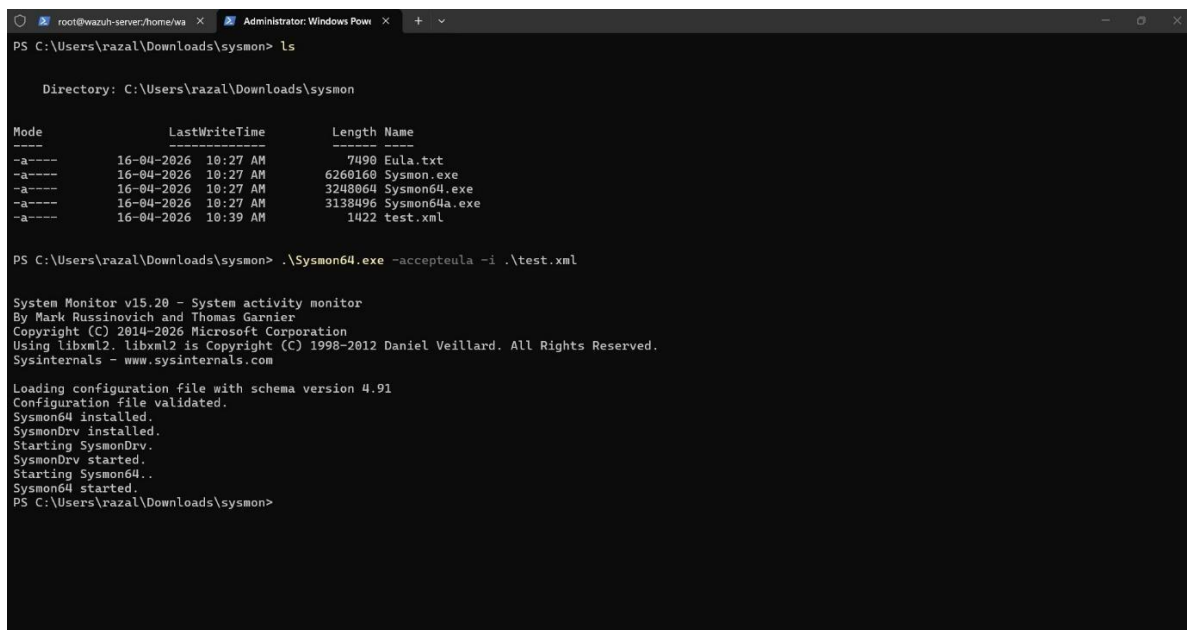


Screenshots

Downloading and Configuring Sysmon in Windows 11



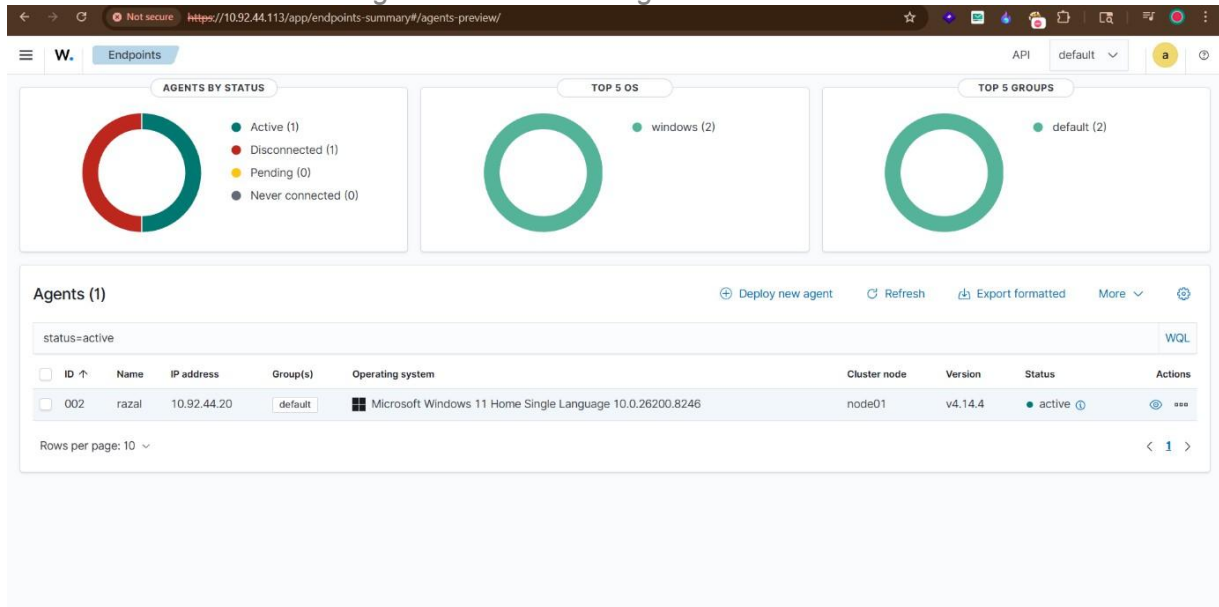
Sysmon Installation Command and Configuration Validated



Setting up Wazuh Agent on Windows 11

The Wazuh agent was installed on Windows 11 and configured to connect to the Wazuh Manager at 10.92.44.113. The ossec.conf was configured to forward Sysmon and Windows Security logs

Wazuh Agent Service Running on Windows 11



```
GNU nano 8.3 ossec.conf Modified
<command>
  <name>netsh</name>
  <executable>netsh.exe</executable>
  <timeout_allowed>yes</timeout_allowed>
</command>

<!--
<active-response>
  active-response options here
</active-response>
-->

<!-- Log analysis -->
<localfile>
  <log_format>command</log_format>
  <command>df -P</command>
  <frequency>360</frequency>
</localfile>

<!-- Sysmon Analysis -->
<localfile>
  <location>Microsoft-Windows-Sysmon/Operational</location>
  <log_format>eventchannel</log_format>
</localfile>

<localfile>
  <log_format>full_command</log_format>
  <command>netstat -tulnp | sed 's/[[:alnum:]]\+ \+ [[:digit:]]\+ \+ [[:digit:]]\+ \+ (.*) : [[:digit:]]\+ \+ ([0-9\.\:]*\+ )\+ [[:digit:]]\+
  <alias>netstat listening ports</alias>
  <frequency>360</frequency>
</localfile>

<localfile>
  <log_format>full_command</log_format>
  <command>last -n 20</command>
  <frequency>360</frequency>
```

Installing TheHive on Ubuntu VM

TheHive 5.2.14 was installed on Ubuntu 24.04 in VirtualBox. Cassandra and Elasticsearch were configured as dependencies

Cassandra Running

```
razal@razal-VirtualBox: ~/Desktop$ sudo systemctl status cassandra
cassandra.service - LSB: distributed storage system for structured data
   Loaded: loaded (/etc/init.d/cassandra; generated)
   Active: active (running) since Fri 2026-05-08 15:51:39 IST; 5min ago
     Docs: man:systemd-sysv-generator(8)
  Process: 1229 ExecStart=/etc/init.d/cassandra start (code=exited, status=0/SUCCESS)
    Tasks: 81 (limit: 9429)
   Memory: 2.4G (peak: 2.4G swap: 780.0K swap peak: 353.6M)
      CPU: 1min 24.458s
  CGroup: /system.slice/cassandra.service
          └─2483 /usr/bin/java -ea -da:net.openhtf... -XX:+UseThreadPriorities -XX:+HeapDumpOnOutOfMemoryError -Xss256k -XX:+AlwaysPreTouch -XX:-UseBiasedLocking -XX:+UseTLAB -XX:

May 08 15:51:38 razal-VirtualBox systemd[1]: Starting cassandra.service - LSB: distributed storage system for structured data...
May 08 15:51:39 razal-VirtualBox systemd[1]: Started cassandra.service - LSB: distributed storage system for structured data.
lines 1-13/13 (END)
```

Elasticsearch Active and Running

```
lines 1-15/15 (END)
razal@razal-VirtualBox: ~/Desktop$ sudo systemctl status elasticsearch
● elasticsearch.service - Elasticsearch
   Loaded: loaded (/usr/lib/systemd/system/elasticsearch.service; enabled; preset: enabled)
   Active: active (running) since Fri 2026-05-08 15:53:30 IST; 4min 50s ago
     Docs: https://www.elastic.co
  Main PID: 1232 (java)
    Tasks: 74 (limit: 9429)
   Memory: 2.2G (peak: 4.2G swap: 2.2G swap peak: 2.2G)
      CPU: 2min 11.647s
  CGroup: /system.slice/elasticsearch.service
          └─1232 /usr/share/elasticsearch/jdk/bin/java -Xshare:auto -Des.networkaddress.cache.ttl=60 -Des.networkaddress.cache.negative.ttl=10 -XX:+AlwaysPreTouch -Xss1m -Djava.awt
          └─2400 /usr/share/elasticsearch/modules/x-pack-ml/platform/linux-x86_64/bin/controller

May 08 15:51:38 razal-VirtualBox systemd[1]: Starting elasticsearch.service - Elasticsearch...
May 08 15:52:04 razal-VirtualBox systemd-entrypoint[1232]: May 08, 2026 3:52:04 PM sun.util.locale.provider.LocaleProviderAdapter <clinit>
May 08 15:52:04 razal-VirtualBox systemd-entrypoint[1232]: WARNING: COMPAT locale provider will be removed in a future release
May 08 15:53:30 razal-VirtualBox systemd[1]: Started elasticsearch.service - Elasticsearch.
lines 1-16/16 (END)
```

TheHive Service Active and Running

```
razal@razal-VirtualBox: ~/Desktop$ sudo systemctl status thehive
thehive.service - Scalable, Open Source and Free Security Incident Response Solutions
   Loaded: loaded (/usr/lib/systemd/system/thehive.service; enabled; preset: enabled)
   Active: active (running) since Fri 2026-05-08 15:51:38 IST; 8min ago
     Docs: https://thehive-project.org
  Main PID: 1239 (java)
    Tasks: 79 (limit: 9429)
   Memory: 920.6M (peak: 951.3M swap: 60.8M swap peak: 81.8M)
      CPU: 1min 53.504s
  CGroup: /system.slice/thehive.service
          └─1239 java -Dfile.encoding=UTF-8 -Dconfig.file=/etc/thehive/application.conf -Dlogger.file=/etc/thehive/logback.xml -Dpidfile.path=/dev/null -cp /opt/thehive/lib/org.the

May 08 15:51:38 razal-VirtualBox systemd[1]: Started thehive.service - Scalable, Open Source and Free Security Incident Response Solutions.
lines 1-12/12 (END)
```

```
razal@razal-VirtualBox: ~/Desktop$ nodetool status
Datacenter: datacenter1
=====
Status=Up/Down
-- State=Normal/Leaving/Joining/Moving
-- Address Load Tokens Owns (effective) Host ID Rack
UN 127.0.0.1 3.94 MiB 16 100.0% e3ce10ea-e7c4-4497-80a9-a3b342878ae0 rack1

razal@razal-VirtualBox: ~/Desktop$
```

The screenshot displays the 'Users' management interface in TheHive Admin. The top navigation bar shows the title 'Users' and a language selector set to 'ENGLISH (UK)'. The main content area features a table with the following columns: NAME, LOGIN, ORGANISATIONS, MFA, CREATED BY, and DATES. Two users are listed:

NAME	LOGIN	ORGANISATIONS	MFA	CREATED BY	DATES
org-admin	admin@thehive.local	A D		TheHive system user	C: 28/04/2026 18:05 U: 03/05/2026 07:04
Shuffle Bot	shuffle@default.local	Preview D		org-admin	C: 04/05/2026 17:50

The bottom of the interface shows a pagination bar indicating '0 - 2 of 2' items and a 'Show 30' dropdown menu.

Creating Firewall Rules

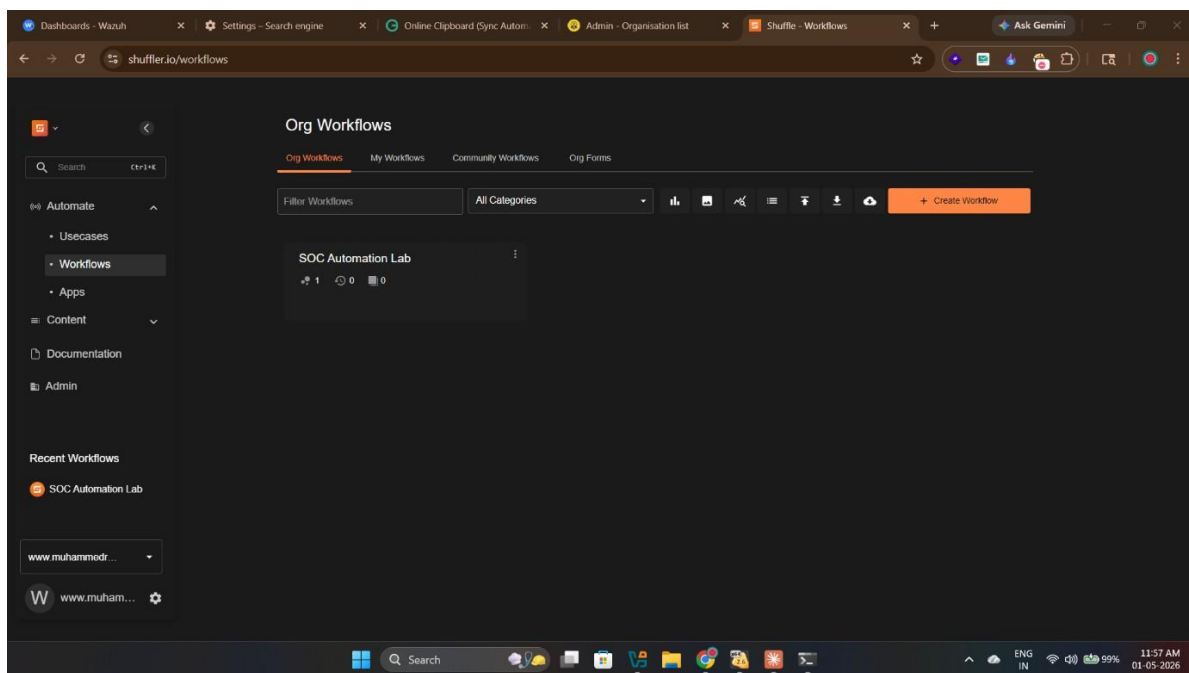
UFW firewall rules were configured on the Ubuntu VM to allow inbound connections on port 9000 for TheHive.

```
razal@razal-VirtualBox:~/Desktop$ sudo ufw allow 9000/tcp
Rule added
Rule added (v6)
razal@razal-VirtualBox:~/Desktop$ sudo ufw allow 9200/tcp
Rule added
Rule added (v6)
razal@razal-VirtualBox:~/Desktop$ sudo ufw status
Status: active

To Action From
--
9997/tcp ALLOW Anywhere
9000/tcp ALLOW Anywhere
9200/tcp ALLOW Anywhere
9997/tcp (v6) ALLOW Anywhere (v6)
9000/tcp (v6) ALLOW Anywhere (v6)
9200/tcp (v6) ALLOW Anywhere (v6)
```

Setting up Shuffle

A Shuffle account was created and the SOC Automation Lab workflow was built at shuffler.io.



shuffler.io/workflows/afa7d86e-ef3a-4ac6-83c8-78120ff1e0e?execution_id=ab49703b-a1e0-4fb8-868d-87d51b26d918&node=b0014346-fdb2-477...

Search apps, triggers...

Popular Actions

Triggers

Your Apps

AI Agent
Singul
Shuffle Tools
Http
Outlook Office365
VirusTotal v3
TheHive

AppsVars

SOC Automation Lab

BuildDebug

Runtime Location (Cloud)
DefaultCloud

Http 1

Wazuh-Alerts

Shu256regex

VirusTotal v3 1

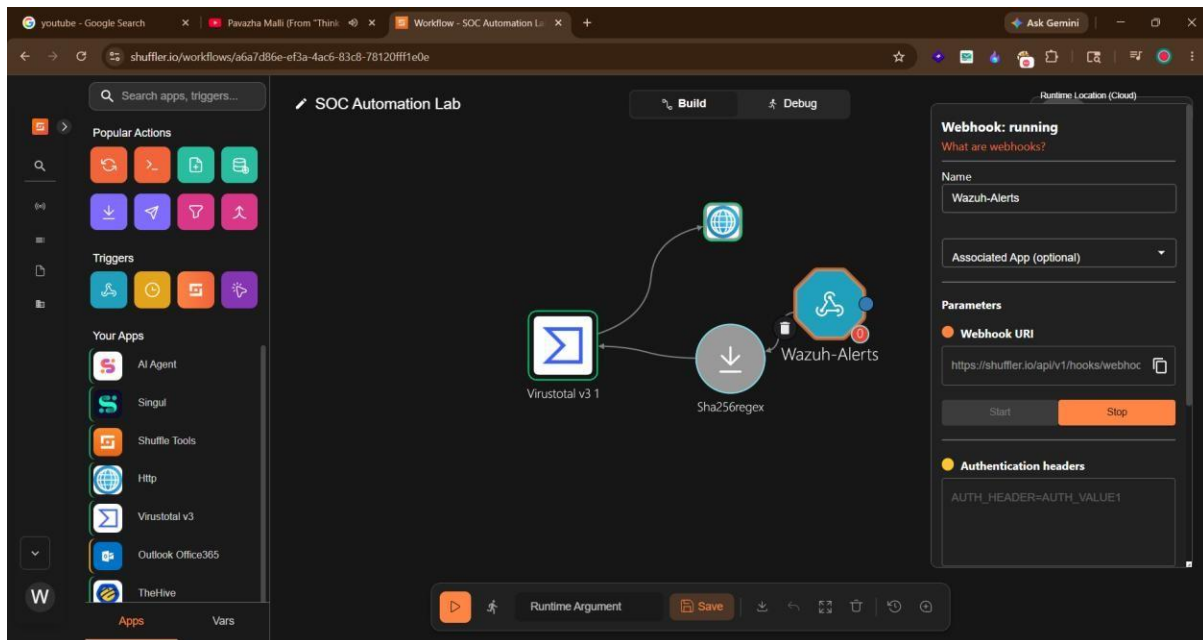
▶

["rule":{"id":"100002","level"

Save

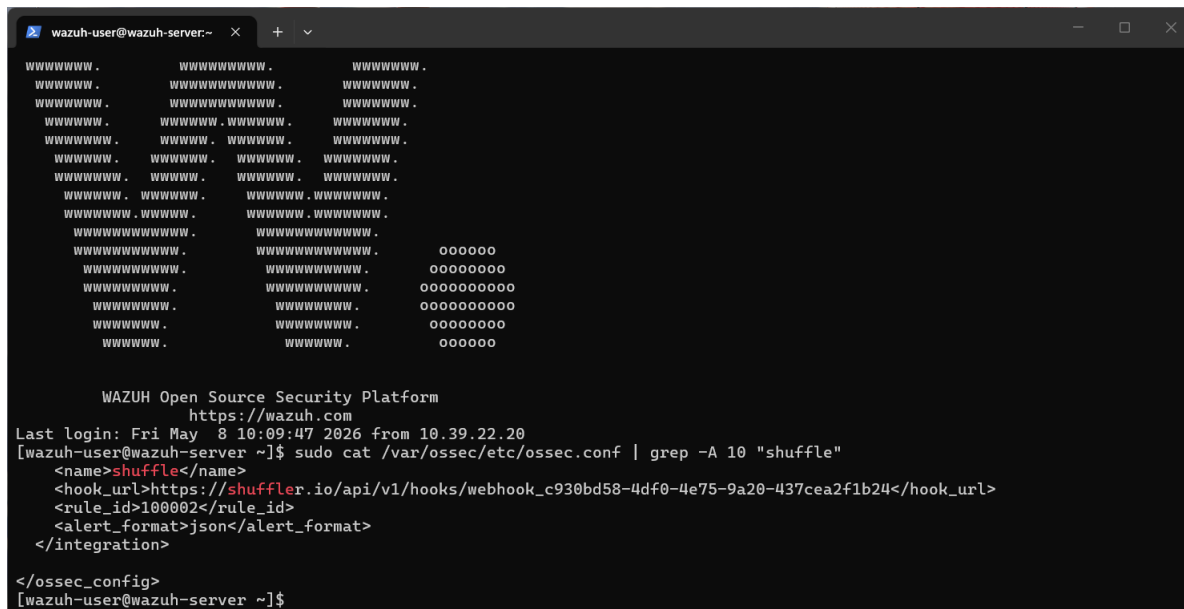
Adding Webhook Trigger

A Wazuh-Alerts webhook trigger was added to the workflow and started to receive alerts from Wazuh Manager



Configuring Wazuh to Connect to Shuffle

The ossec.conf file on the Wazuh Manager was modified to add the Shuffle integration block with the webhook URL and rule ID 100002



Creating Custom Mimikatz Detection Rule

A custom Wazuh rule (ID 100002) was created in local_rules.xml to detect Mimikatz execution based on the originalFileName field.

```
wazuh-user@wazuh-server:~$ cat local_rules.xml
<!-- Modify it at your will. -->
<!-- Copyright (C) 2015, Wazuh Inc. -->

<!-- Example -->
<group name="local,syslog,sshd,">

  <!--
  Dec 10 01:02:02 host sshd[1234]: Failed none for root from 1.1.1.1 port 1066 ssh2
  -->
  <rule id="100001" level="5">
    <if_sid>5716</if_sid>
    <srcip>1.1.1.1</srcip>
    <description>sshd: authentication failed from IP 1.1.1.1.</description>
    <group>authentication_failed,pci_dss_10.2.4,pci_dss_10.2.5,</group>
  </rule>
</group>

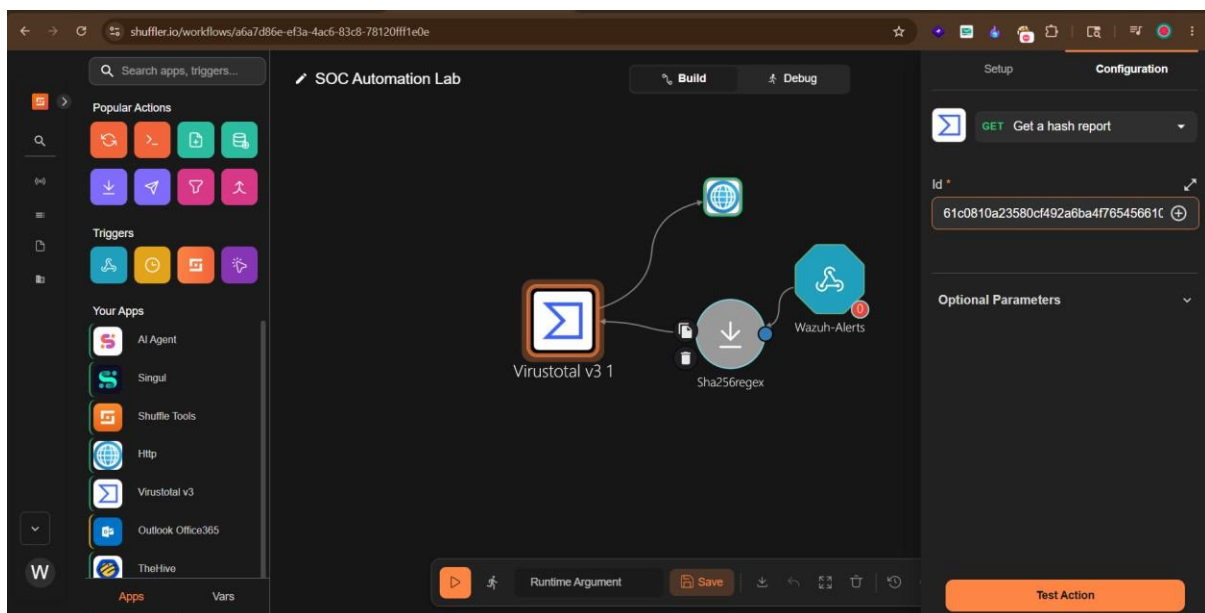
<group name="windows,sysmon,custom">
  <rule id="100301" level="12">
    <if_sid>61683</if_sid>
    <field name="win.eventdata.image" type="pcre2">(?!powershell.exe</field>
    <field name="win.eventdata.commandLine" type="pcre2">(?!s-(enc|encodedcommand)\b</field>
    <description>⚠ Sysmon: Encoded PowerShell command detected</description>
    <mitre>
      <id>T1059.001</id>
      <id>T1027</id>
    </mitre>
  </rule>
</group>

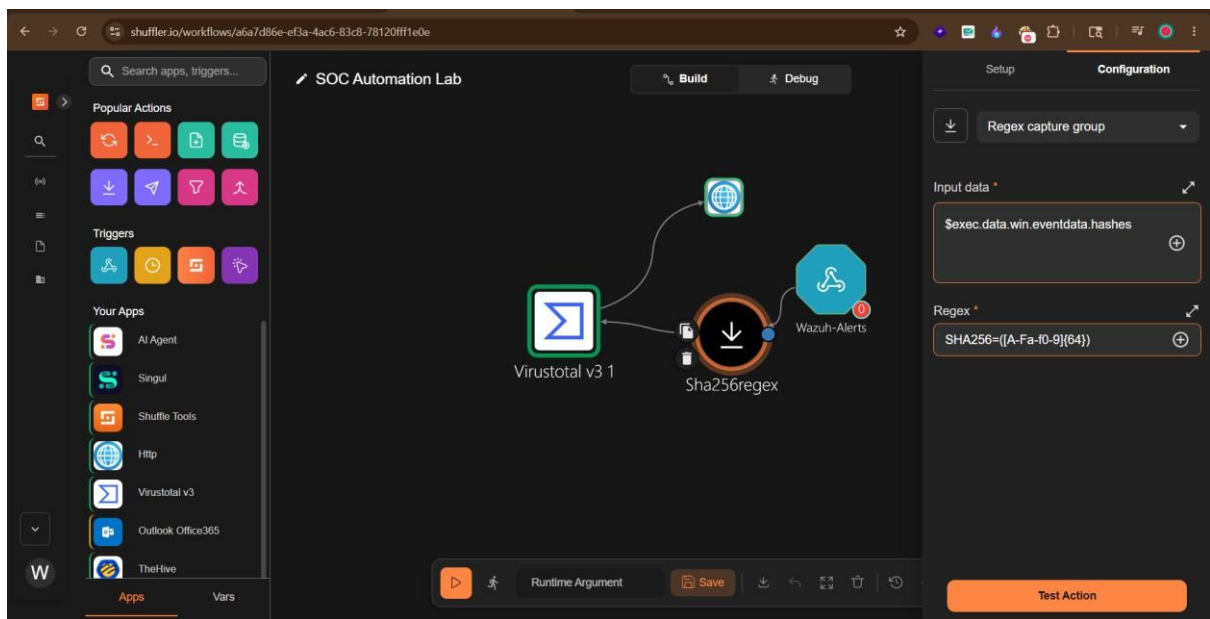
<group name="sysmon,">
  <rule id="100002" level="15">
    <if_group>sysmon_event1</if_group>
    <field name="win.eventdata.originalFileName" type="pcre2">(?!mimikatz\.</field>
    <description>⚠ Mimikatz Usage Detected</description>
    <mitre>
      <id>T1003</id>
    </mitre>
  </rule>
</group>

[wazuh-user@wazuh-server ~]$
```

Integrating VirusTotal

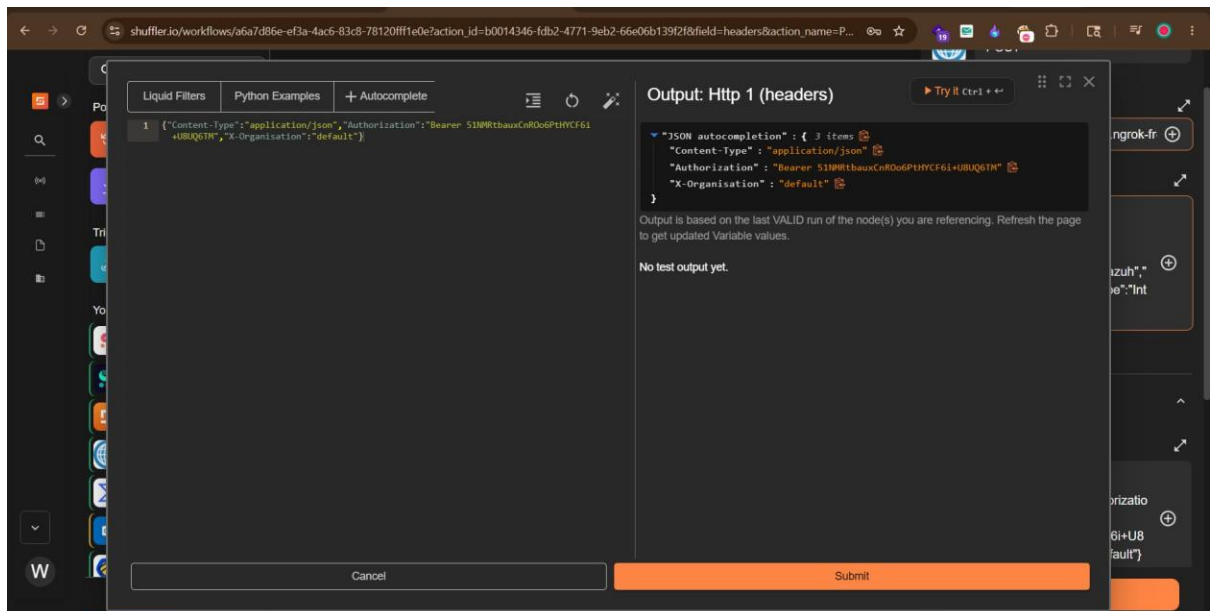
VirusTotal v3 was added to the Shuffle workflow to enrich the SHA256 hash extracted from alerts

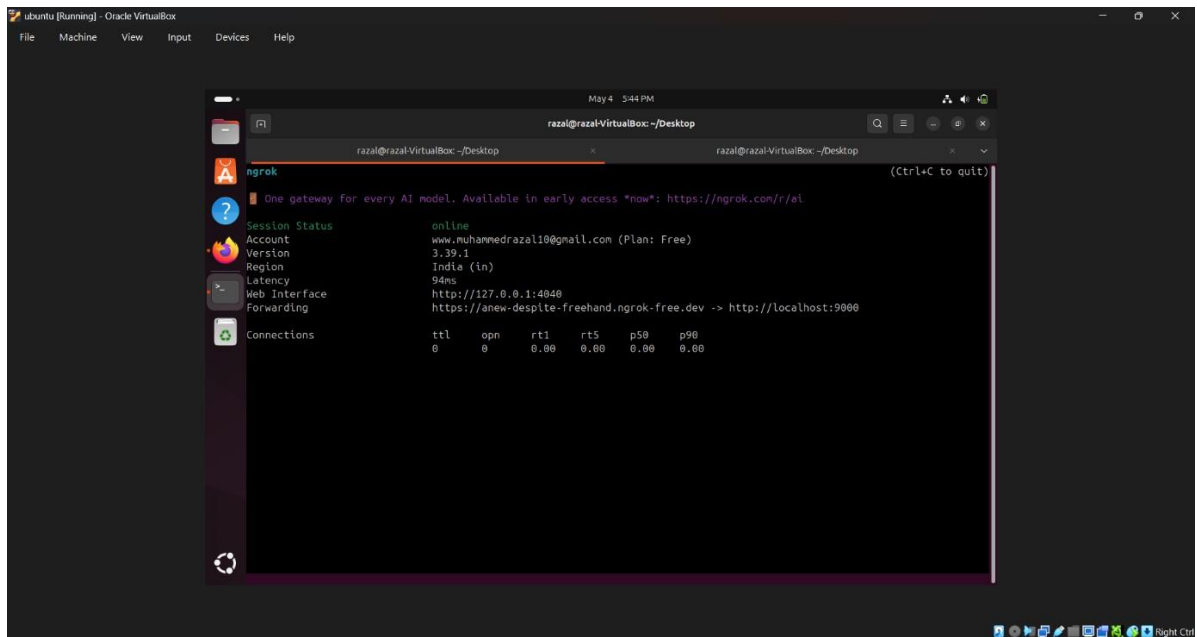
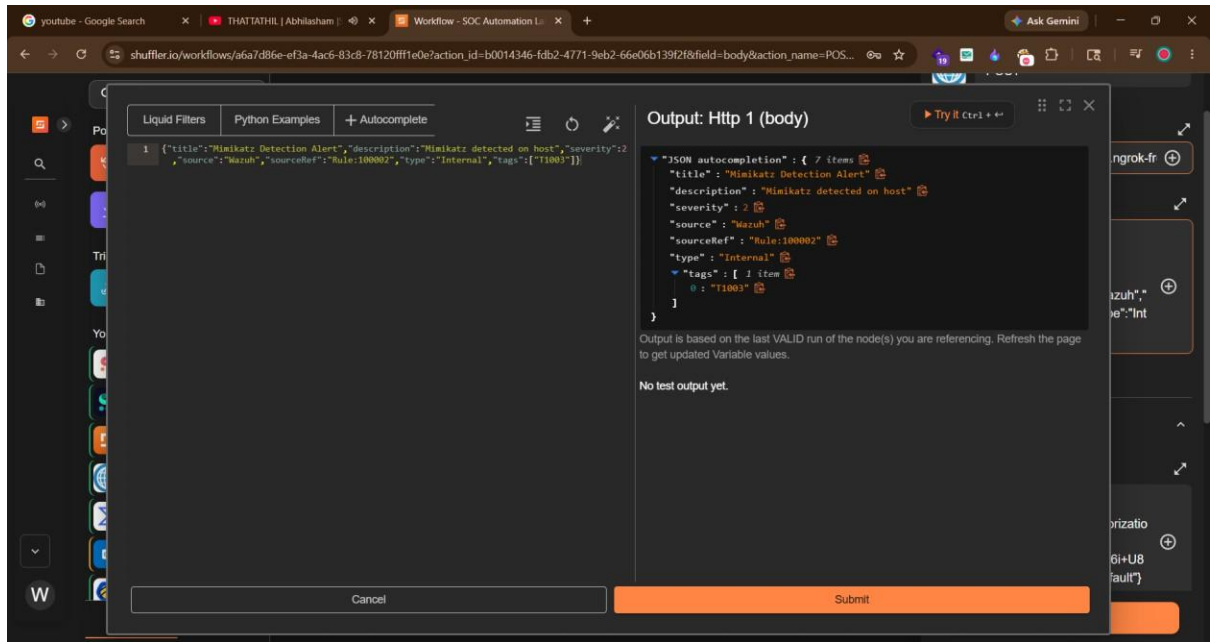




Configuring Shuffle to Work with TheHive

TheHive was added to the workflow with authentication using the org-admin API key and the ngrok public URL

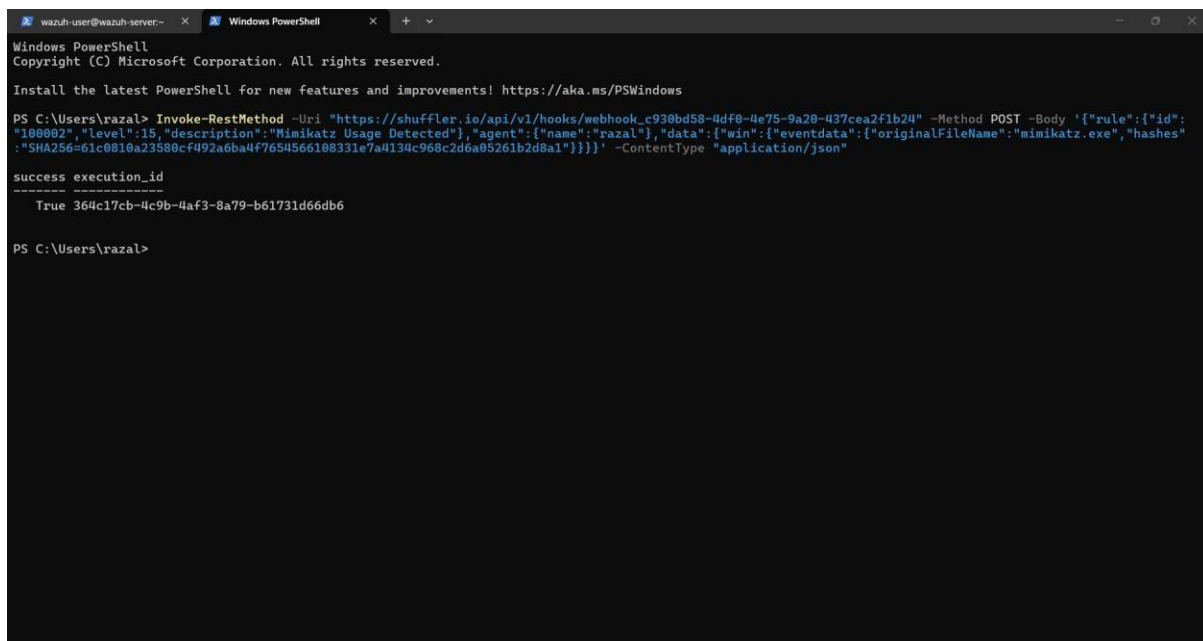




Testing and Results

Test Execution

A simulated Mimikatz alert was sent directly to the Shuffle webhook using PowerShell to test the complete pipeline without executing actual malware on the system.



```
wazuh-user@wazuh-server:~$ Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\razal> Invoke-RestMethod -Uri "https://shuffler.io/api/v1/hooks/webhook_c938bd58-4df0-4e75-9a20-437cea2f1b24" -Method POST -Body '{"rule":{"id":
"100002","level":15,"description":"Mimikatz Usage Detected"},"agent":{"name":"razal"},"data":{"win":{"eventdata":{"originalFileName":"mimikatz.exe","hashes"
:"SHA256=61c0810a23580cf492a6ba4f7654566108331e7a4134c968c2d6a05261b2d8a1"}}}}' -ContentType "application/json"

success execution_id
-----
True 364c17cb-4c9b-4af3-8a79-b61731d66db6

PS C:\Users\razal>
```

Shuffle Workflow Execution

The Shuffle Debug tab confirmed the workflow executed successfully with Status: FINISHED across all nodes.

The screenshot displays the Shuffle Workflow Execution interface. The main workspace shows a workflow diagram with three nodes: 'Http 1' (top), 'VirusTotal v3' (bottom left), and 'Sha256regex' (bottom right). The 'Http 1' node is connected to both 'VirusTotal v3' and 'Sha256regex'. The 'VirusTotal v3' node is connected to 'Sha256regex'. The interface includes a left sidebar with 'Popular Actions', 'Triggers', and 'Your Apps'. The right sidebar shows the 'Details' tab for the workflow, indicating it is 'FINISHED' with a 'webhook' source. The 'Details' tab also displays the workflow's JSON configuration, which includes a rule for detecting Mimikatz usage.

```
{
  "$exec": {
    "rule": {
      "id": "100002",
      "level": 15,
      "description": "Mimikatz Usage Detected"
    },
    "agent": {
      "name": "razal"
    },
    "data": {
      "win": {
        "eventdata": {
          "originalFileName": "mimikatz.exe"
        }
      }
    }
  }
}
```

The screenshot shows the configuration for the 'sha256regex' node, which is a 'regex_capture_group' type. The configuration is displayed in a JSON format within a dark-themed editor. The configuration includes a 'success' field set to 'true', a 'group_0' field with a regex pattern '[...]', and a 'found' field set to 'true'.

```
{
  "$sha256regex": {
    "success": true,
    "group_0": [...],
    "found": true
  }
}
```

shuffler.io/workflows/a6a7d86e-ef3a-4ac6-83c8-78120fff1e0e?execution_id=b8cf35c1-30f5-4a35-adba-a2263e7e874b

Build

Debug

Runtime Location (Cloud)
Default Cloud

Search apps, triggers...

Popular Actions

Triggers

Your Apps

AI Agent

Singul

Shuffle Tools

Http

VirusTotal v3

Outlook Office365

TheHive

AppsVars

SOC Automation Lab

VirusTotal v3 1

Http 1

ShodanScreex

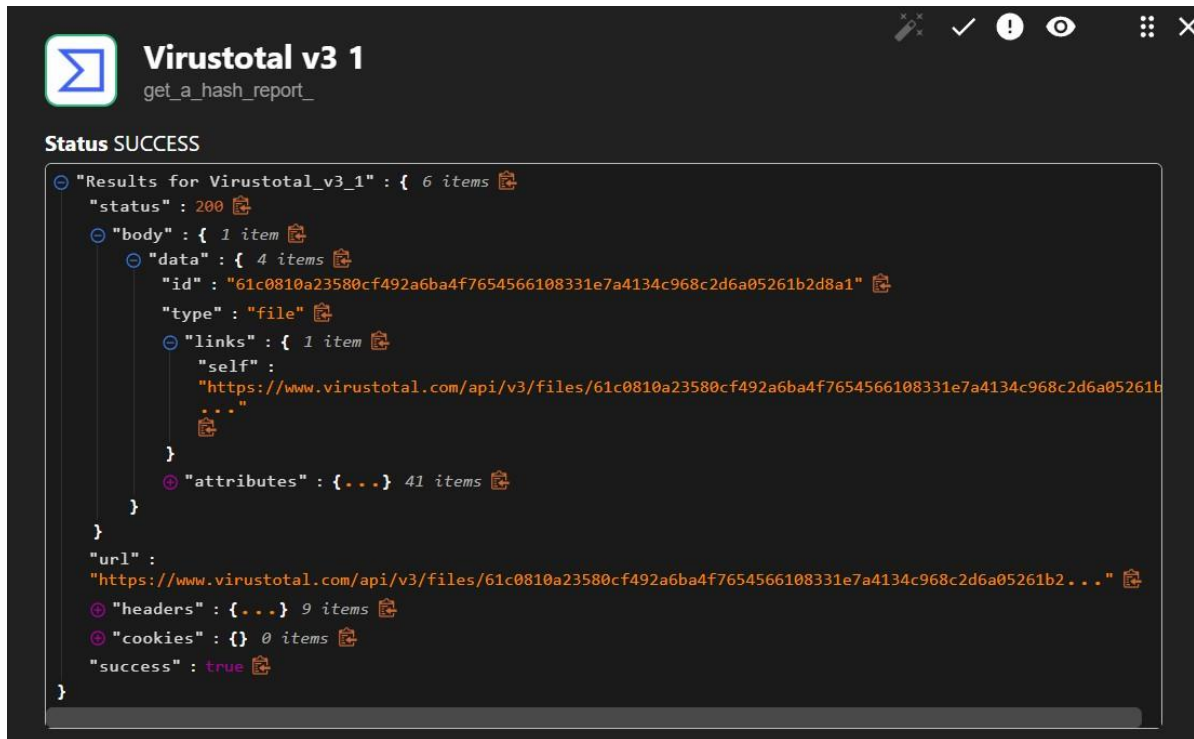
Wazuh-Alerts

Runtime Argument

Save

VirusTotal Result

VirusTotal returned a Status 200 response confirming the hash was identified in its database with multiple malicious detections.



The screenshot shows a web browser window with the title "VirusTotal v3 1" and a tab labeled "get_a_hash_report_". The page displays a "Status SUCCESS" message. Below this, a JSON viewer shows the following data:

```
{
  "Results for Virustotal_v3_1": {
    "status": 200,
    "body": {
      "data": {
        "id": "61c0810a23580cf492a6ba4f7654566108331e7a4134c968c2d6a05261b2d8a1",
        "type": "file",
        "links": {
          "self": "https://www.virustotal.com/api/v3/files/61c0810a23580cf492a6ba4f7654566108331e7a4134c968c2d6a05261b2d8a1"
        },
        "attributes": {
          // 41 items
        }
      }
    },
    "url": "https://www.virustotal.com/api/v3/files/61c0810a23580cf492a6ba4f7654566108331e7a4134c968c2d6a05261b2d8a1",
    "headers": {
      // 9 items
    },
    "cookies": {
      // 0 items
    },
    "success": true
  }
}
```

TheHive Alert Created

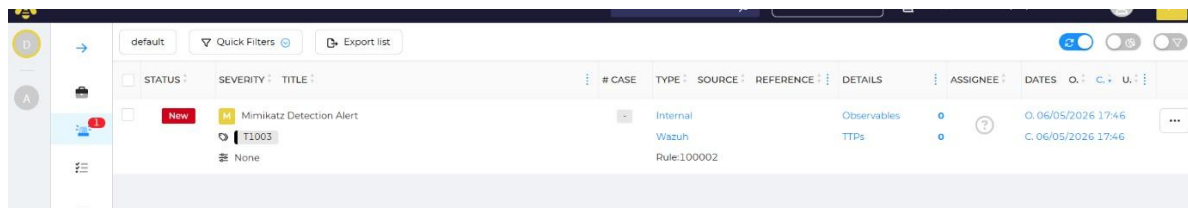
TheHive returned Status 201 confirming the alert was successfully created in the case management system.



Alert in TheHive Dashboard

The Mimikatz Detection Alert appeared in TheHive dashboard with the following confirmed details:

Component	Details
Title	Mimikatz Detection Alert
Status	New
Severity	Medium (2)
Source	Wazuh
Source Reference	Rule:100002
Type	Internal
Tags	T1003
Created	03/05/2026 07:10



The screenshot shows the TheHive dashboard interface. At the top, there are tabs for 'default', 'Quick Filters', and 'Export list'. Below this is a table with columns: STATUS, SEVERITY, TITLE, # CASE, TYPE, SOURCE, REFERENCE, DETAILS, ASSIGNEE, DATES, O., C., U., and a menu icon. The table contains one row for the 'Mimikatz Detection Alert'. The status is 'New' (indicated by a red tag), severity is 'Medium' (indicated by a yellow tag), and title is 'Mimikatz Detection Alert'. The source is 'Wazuh' and the reference is 'Rule:100002'. The type is 'Internal'. The details section shows 'Observables' and 'TTPs'. The dates are 'O. 06/05/2026 17:46' and 'C. 06/05/2026 17:46'. The assignee is empty, and there is a question mark icon next to it. The table also shows a 'None' tag for the severity and a 'T1003' tag for the tags.

STATUS	SEVERITY	TITLE	# CASE	TYPE	SOURCE	REFERENCE	DETAILS	ASSIGNEE	DATES	O.	C.	U.	
New	Medium	Mimikatz Detection Alert	1	Internal	Wazuh	Rule:100002	Observables TTPs		0. 06/05/2026 17:46 C. 06/05/2026 17:46				

Conclusion

This SOC automation project represents a significant stride towards enhancing security operations efficiency and effectiveness. By integrating Wazuh, TheHive, Shuffle, and Sysmon, a robust and automated security ecosystem was successfully created, capable of detecting, responding to, and managing potential threats in real-time.

The project achieved all its primary objectives:

- Wazuh successfully collected and analyzed security events from Windows 11 via Sysmon integration.
- Custom detection rules were created to identify Mimikatz credential dumping attacks.
- Shuffle automated the complete workflow from alert detection through IOC enrichment to case creation.
- VirusTotal successfully enriched IOCs with threat intelligence confirming malicious file hashes.
- TheHive automatically received structured alerts enabling SOC analyst investigation and response.

This SOC automation project serves as a testament to the power of open-source tools and collaborative efforts in building robust and efficient security solutions. It empowers students to gain hands-on experience with cutting-edge security technologies and prepares them for future careers in cybersecurity.

